FACULTAD DE CIENCIAS Y TECNOLOGÍAS DE LA INFORMACIÓN

UNIVERSIDAD AUTÓNOMA DE GUERRERO

Integrantes:

- **Gadi Jeani del Carmen Memije**
- **Roberto Adan Patino Rayo**

Proyecto Integrador 2

Gestión de reserva de asientos de cine con matrices

Materia: Estructura de datos

Repositorio: https://github.com/Roberto8322/Proyecto-Integrador-2-cine

13/11/2026

2. Integrantes

Gadi Jeani del Carmen Memije

Roberto Adan Patino Rayo

Los dos trabajamos en todas las etapas del proyecto: el análisis, el diseño, el código y la documentación. Cada quien subió sus avances al mismo repositorio de GitHub, con sus propios commits, y las partes se repartieron por día según lo que tocaba entregar.

Etapa	Qué se hizo	Quién
Análisis del problema	Requerimientos, entradas, procesos, salidas y casos de uso	Los dos
Algoritmo y pseudocódigo	Pasos generales del programa	Los dos
Diagramas de flujo y diseño	Un diagrama por función, diseño modular y diseño de la interfaz	Los dos

Código en C#	Menú, sala, reservar, liberar, consultar y estadísticas	Los dos
Comentarios y revisión de nombres	Se comentó el código y se revisaron los nombres de funciones	Los dos
Documentación	La mitad cada uno	Los dos

3. Introducción

En este proyecto hicimos un programa para controlar los asientos de una sala de cine. Está hecho en C#, como aplicación de consola, y se ejecuta en .NET. La sala se guarda en una matriz de 10 filas por 10 columnas, que da un total de 100 asientos.

Cada asiento se marca con una letra. La O quiere decir que el asiento está libre y la X que está ocupado. Las filas se identifican con letras de la A a la J y las columnas con números del 0 al 9, así que un asiento se nombra con una letra y un número, por ejemplo C5.

El programa tiene un menú principal con estas opciones:

- Salir
- Mostrar sala de asientos
- Consultar disponibilidad
- Reservar asientos
- Liberar asiento
- Estadística de ocupación

Al reservar, liberar o consultar, el usuario puede indicar cuántos asientos quiere manejar y después escribe la fila y la columna de cada uno. Cuando libera asientos, el programa le pregunta el motivo: salud, viaje, cambio de planes u otro. En las estadísticas se muestra cuántos asientos están ocupados, cuántos están libres y el porcentaje de ocupación de la sala.

El proyecto lo trabajamos los dos con un solo repositorio de GitHub. Cada uno fue subiendo su parte día por día, y el historial de commits muestra cómo fue avanzando el trabajo.

4. Planteamiento del problema

Un cine necesita llevar el control de los asientos de sus salas. Si esto se hace a mano o en papel, se pueden cometer errores: vender dos veces el mismo asiento, no saber qué lugares siguen libres o no tener claro cuántas personas hay en la sala.

Se necesita un programa que haga lo siguiente:

- Mostrar la sala completa, con los asientos libres y los ocupados.
- Permitir reservar un asiento, y avisar si ya estaba ocupado.
- Permitir liberar un asiento, y avisar si ya estaba libre.
- Permitir consultar si un asiento está libre u ocupado.
- Contar los asientos ocupados y los libres.
- Indicar qué porcentaje de la sala está ocupado.
- Avisar cuando el usuario escribe algo que no es válido, como una letra de fila que no existe o un texto donde va un número.

Para guardar la sala se pidió usar una matriz, porque los asientos de un cine ya están ordenados en filas y columnas.

5. Objetivos

Objetivo general

Hacer un programa en C# que administre los asientos de una sala de cine usando una matriz, para que se puedan reservar, liberar y consultar lugares y ver cuánto está ocupada la sala.

Objetivos específicos

- Representar la sala con una matriz de caracteres de 10 filas (A a J) por 10 columnas (0 a 9).
- Mostrar la sala en pantalla con los asientos libres (O) y los ocupados (X).
- Reservar asientos, revisando antes que estén libres.
- Liberar asientos, guardando el motivo por el que se devuelven.
- Consultar si un asiento está libre u ocupado.
- Contar los asientos ocupados y libres, y calcular el porcentaje de ocupación.
- Validar lo que escribe el usuario: letras de fila de la A a la J, columnas de 0 a 9 y cantidades dentro del rango permitido.
- Dividir el programa en funciones, cada una con una sola tarea.
- Usar Git y GitHub para trabajar en equipo y dejar el historial del avance.

6. Análisis

6.1 Entradas

- La opción del menú principal (0 a 5).
- La cantidad de asientos que se quieren reservar, liberar o consultar.
- La letra de la fila (A a J).
- El número de la columna (0 a 9).
- El motivo de liberación del asiento (salud, viaje, cambio de planes u otro, con opción de escribirlo).

6.2 Procesos

- Llenar toda la matriz con O cuando empieza el programa.
- Repetir el menú hasta que el usuario elija salir.
- Revisar que la cantidad esté entre los valores permitidos: de 1 a 10 para reservar y liberar, y de 1 a 100 para consultar.
- Convertir la letra de la fila en su número (A es 0, B es 1, y así hasta J, que es 9).
- Revisar que la columna sea un número entre 0 y 9.
- Leer qué hay en la posición pedida, para saber si el asiento está libre u ocupado.
- Cambiar la O por X al reservar, y la X por O al liberar.
- Recorrer toda la matriz y contar cuántos asientos tienen X y cuántos tienen O.
- Calcular el porcentaje de ocupación: ocupados entre 100, por 100.

6.3 Salidas

- El menú principal.
- La sala impresa con letras en las filas y números en las columnas.
- Mensajes de reserva realizada, asiento liberado, asiento ocupado o asiento ya libre.
- El estado del asiento consultado (libre u ocupado).
- El total de asientos ocupados, el total de libres y el porcentaje de ocupación.
- Mensajes de error cuando un dato no es válido.

6.4 Requerimientos funcionales

Clave	Requerimiento
RF-01	El sistema mostrará la sala completa cuando el usuario lo pida.
RF-02	El sistema pedirá fila y columna para reservar un asiento.
RF-03	El sistema revisará que el asiento esté libre antes de reservarlo.
RF-04	El sistema cambiará el asiento a ocupado cuando se confirme la reserva.
RF-05	El sistema permitirá liberar un asiento ocupado, indicando fila y columna.
RF-06	El sistema permitirá consultar si un asiento está libre u ocupado.
RF-07	El sistema contará los asientos ocupados y los libres.
RF-08	El sistema calculará y mostrará el porcentaje de ocupación.

6.5 Requerimientos no funcionales

Clave	Requerimiento
RNF-01	La interfaz debe mostrar mensajes claros cuando el usuario se equivoque (fila fuera de rango, columna que no es número, asiento ocupado).
RNF-02	La respuesta en pantalla debe ser inmediata.

RNF-03	El programa se hace en C#, sobre .NET, como aplicación de consola.

6.6 Estructura de datos

Para guardar la sala usamos una matriz de caracteres de 10 por 10 (`char[10, 10]`).

Elegimos la matriz porque una sala de cine ya está organizada en filas y columnas, y la matriz se parece mucho a eso. Con ella se llega a cualquier asiento directamente con `sala[fila, columna]`, sin tener que recorrer una lista. También se usa fácil en los ciclos `for`, que sirven para llenar la sala, imprimirla y contar los asientos.

Usamos el tipo `char` porque con una sola letra alcanza para guardar el estado de cada asiento: `'O'` si está libre y `'X'` si está ocupado.

La sala está declarada una sola vez al inicio del programa, y todas las funciones trabajan sobre esa misma matriz.

6.7 Módulos del programa

El programa se dividió en funciones, y cada una hace una sola tarea:

Función	Qué hace
Main	Muestra el menú, lee la opción y llama a la función que toca, hasta que el usuario elige salir.
LlenarSala	Pone una O en los 100 asientos.
MostrarMenu	Imprime las opciones del menú principal.
MostrarSala	Recorre la matriz e imprime la sala.
LeerCoordenadas	Pide la fila y la columna, valida los datos y regresa los dos valores.
ConsultarDisponibilidad	Dice si los asientos que elige el usuario están libres u ocupados.
ReservarAsiento	Cambia a X los asientos libres que elige el usuario.
LiberarAsiento	Pregunta el motivo y cambia a O los asientos ocupados que elige el usuario.
MostrarEstadisticas	Cuenta ocupados y libres, y calcula el porcentaje.

6.8 Alcance y limitaciones

Alcance

- Una sala de cine de 100 asientos (10 filas por 10 columnas).
- La sala se muestra con letras en las filas y números en las columnas.
- Se pueden reservar y liberar hasta 10 asientos cada vez, y consultar hasta 100.
- Se valida la fila, la columna y la cantidad de asientos.
- Al liberar un asiento se elige el motivo: salud, viaje, cambio de planes u otro.
- Se pueden ver las estadísticas de ocupación.

Limitaciones

- El programa funciona solo en consola, sin ventanas ni botones.
- La información no se guarda. Al cerrar el programa, la sala vuelve a quedar toda libre.
- Maneja una sola sala por ejecución.
- No incluye cobro de boletos, formas de pago ni comprobantes.
- El motivo de liberación se pide, pero no se guarda en ningún archivo.

6.9 Casos de uso

Caso 1: Mostrar la sala

- El usuario elige la opción 1 del menú.
- El programa recorre la matriz e imprime los 100 asientos con su estado.

Caso 2: Consultar disponibilidad

- El usuario elige la opción 2 y escribe cuántos asientos quiere consultar.
- Por cada asiento, escribe la fila y la columna.
- El programa dice si el asiento está libre (O) u ocupado (X).

Caso 3: Reservar un asiento libre

- El usuario elige la opción 3 y escribe cuántos asientos quiere reservar.
- Escribe la fila y la columna, por ejemplo C y 5.
- El programa revisa que `sala[C, 5]` tenga una O, la cambia por X y avisa que la reserva se realizó.

Caso 4: Intentar reservar un asiento ocupado

- El usuario escribe la fila y la columna de un asiento que ya tiene X.
- El programa avisa que el asiento ya está ocupado y no cambia nada.

Caso 5: Liberar un asiento ocupado

- El usuario elige la opción 4 y escoge el motivo de liberación.
- Escribe cuántos asientos quiere liberar y la fila y la columna de cada uno.
- El programa cambia la X por O y avisa que el asiento quedó liberado.

Caso 6: Intentar liberar un asiento que ya estaba libre

- El usuario escribe la fila y la columna de un asiento que tiene O.
- El programa avisa que el asiento ya estaba libre.

Caso 7: Ver estadísticas

- El usuario elige la opción 5.
- El programa recorre la matriz, cuenta los asientos ocupados y los libres, y calcula el porcentaje.
- Muestra los tres datos en pantalla.

Caso 8: Escribir un dato que no es válido

- El usuario escribe una letra de fila que no está entre la A y la J, una columna que no es un número del 0 al 9, o una cantidad fuera de rango.
- El programa muestra un mensaje de error y vuelve a pedir el dato.

Caso 9: Elegir una opción que no existe en el menú

- El usuario escribe una opción que no está en el menú.
- El programa avisa que la opción no es válida y regresa al menú.

7. Diseño

El programa se diseñó bajo un enfoque **modular** en C# dentro de una aplicación de consola (Program.cs), dividiendo cada funcionalidad en métodos independientes para facilitar su lectura y mantenimiento:

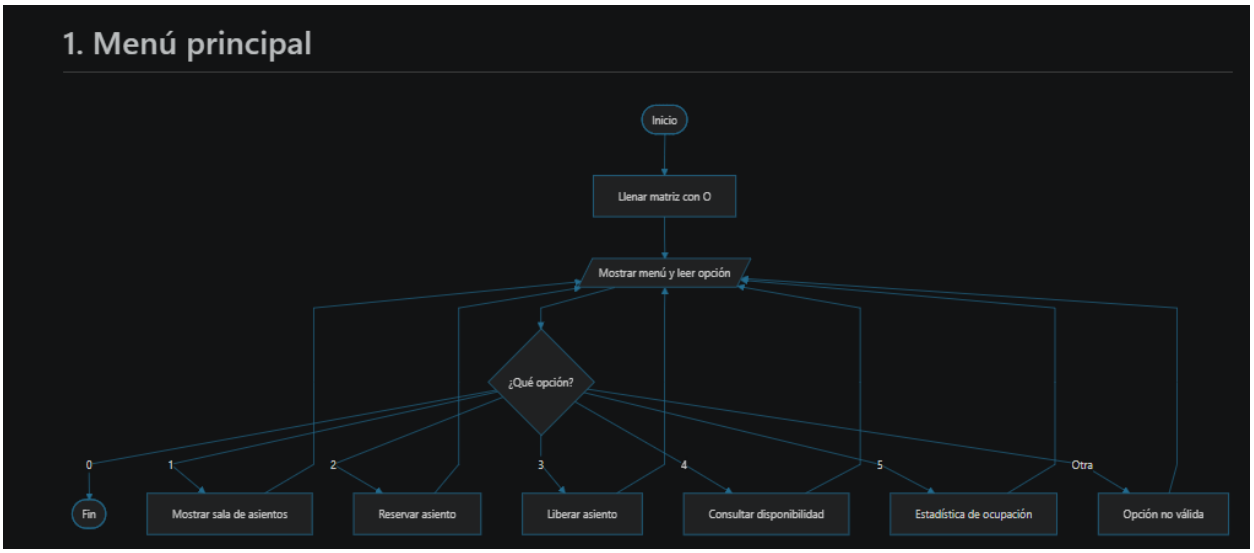
- **Main():** Inicializa la matriz sala llenándola con 'O' y mantiene activo un ciclo do-while que despliega el menú principal y redirige a cada módulo mediante una estructura switch hasta que el usuario selecciona 0.
- **LeerCoordenadas():** Método auxiliar reutilizable que solicita y valida tanto la fila (A-J convertida a índice 0-9 mediante un arreglo de caracteres) como la columna (0-9), devolviendo una tupla (int fila, int columna).
- **MostrarSala():** Recorre la matriz mediante dos ciclos for anidados e imprime en consola el mapa completo de la sala con sus respectivos identificadores.
- **ReservarAsiento():** Invoca a LeerCoordenadas(), verifica si sala[fila, columna] == 'O' y, de ser así, cambia su estado a 'X'.
- **LiberarAsiento():** Invoca a LeerCoordenadas(), verifica si sala[fila, columna] == 'X' y lo restablece a 'O'.

- **ConsultarDisponibilidad():** Invoca a LeerCoordenadas() e informa al usuario si el asiento específico se encuentra libre u ocupado sin alterar su estado.
- **MostrarEstadisticas():** Recorre la matriz contando las ocurrencias de 'X' y 'O', calcula el porcentaje de ocupación con la fórmula (ocupados / 100.0) * 100 y muestra el resumen en pantalla.

8. Diagrama de flujo

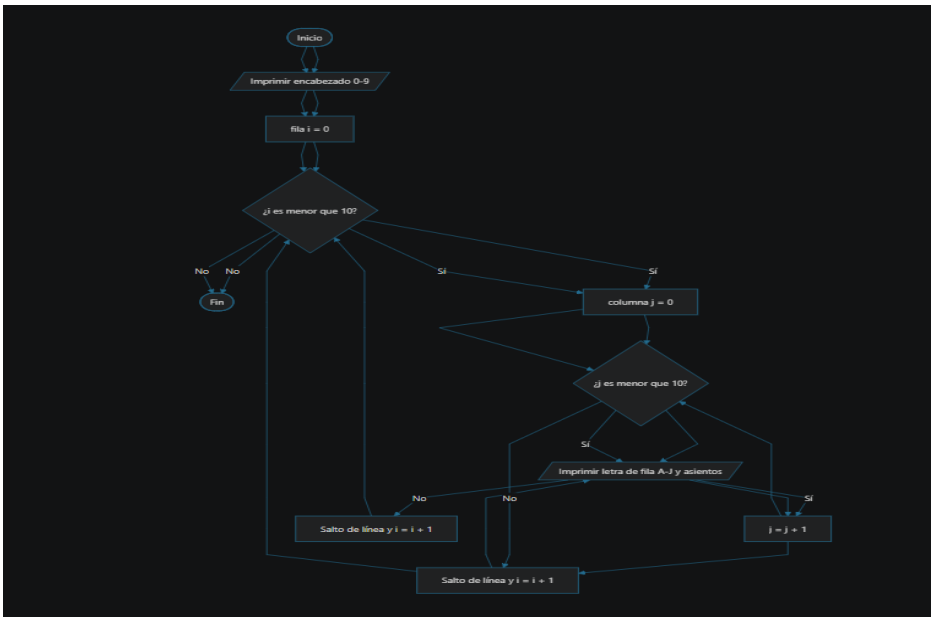
8.1 Menu Principal

- Menú principal (muestra el ciclo de selección de opciones y el retorno automático al menú hasta elegir 0).

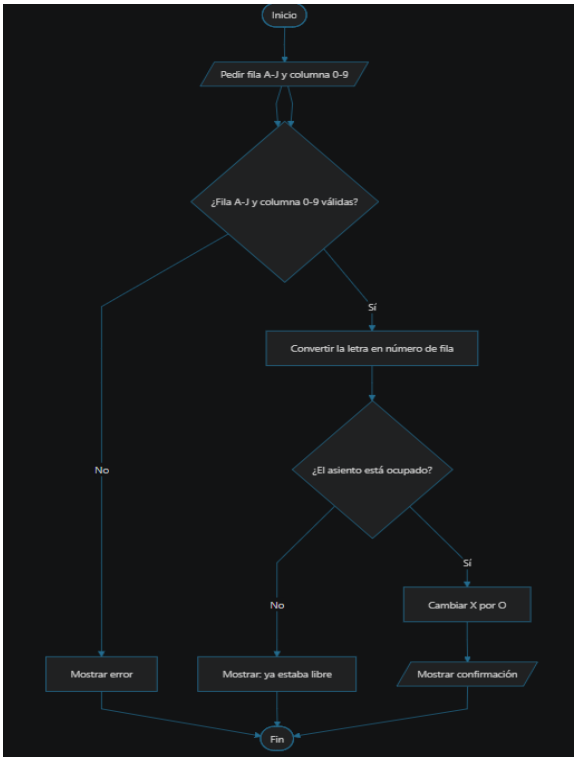


8.2 Mostrar sala

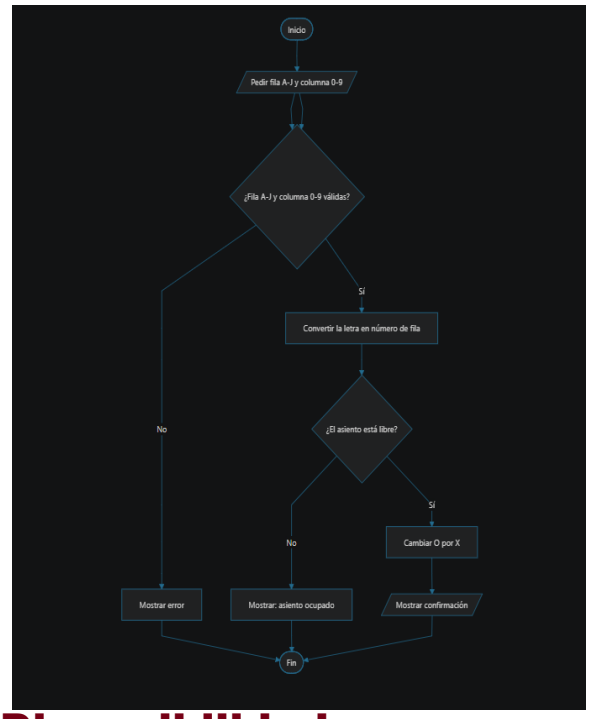
Mostrar sala (muestra el recorrido anidado de filas i y columnas j).



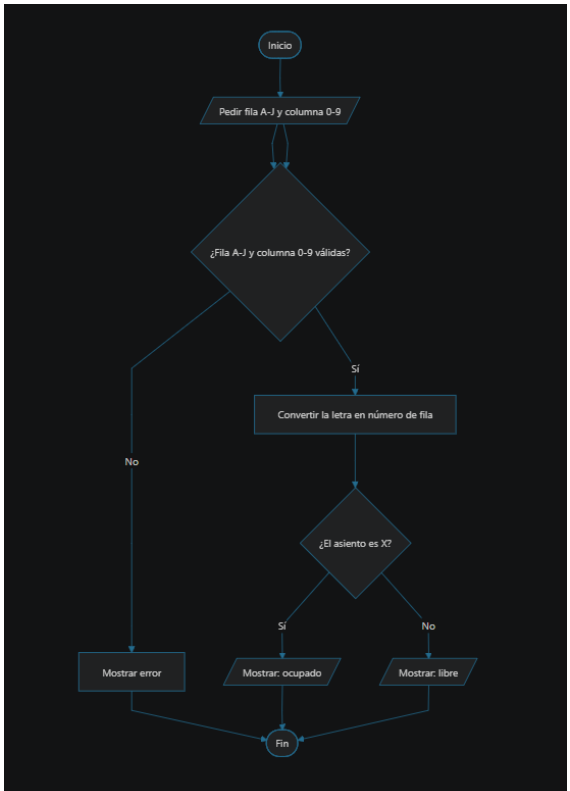
8.3 Reservar asiento



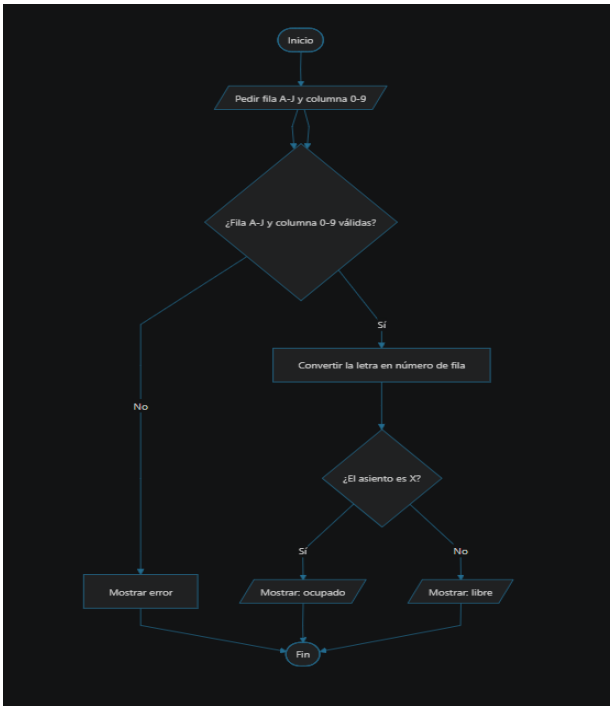
8.4 Liberar asiento



8.5 Consultar Disponibilidad



8.6 Estadísticas



9. Capturas

9.1 Menú principal

```
===== CINE =====
0. Salir
1. Mostrar sala de asientos
2. Consultar disponibilidad
3. Reservar asientos
4. Liberar asiento
5. Estadística de ocupación
Elige una opción:
```

9.2 Mostrar Sala

```
    0 1 2 3 4 5 6 7 8 9
   --------------------
A | O O O O O O O O O O
B | O O O O O O O O O O
C | O O O O O O O O O O
D | O O O O O O O O O O
E | O O O O O O O O O O
F | O O O O O O O O O O
G | O O O O O O O O O O
H | O O O O O O O O O O
I | O O O O O O O O O O
J | O O O O O O O O O O

Presiona Enter para continuar...
```

9.3 Consultar Disponibilidad

```
CONSULTAR DISPONIBILIDAD
Ingrese la cantidad de asientos que desea consultar
a
Cantidad no válida. Intente de nuevo.
2
Ingrese la fila (A-J): s
Fila no válida. Intente de nuevo.
Ingrese la fila (A-J): a
Ingrese la columna (0-9): 1
El asiento seleccionado esta: libre O
Ingrese la fila (A-J): a
Ingrese la columna (0-9): e
Columna no válida. Intente de nuevo.
Ingrese la columna (0-9): 2
El asiento seleccionado esta: libre O

Presiona Enter para continuar...
```

9.4 Reservar Asientos

```
 RESERVAR ASIENTO
Ingrese la cantidad de asientos a reservar
e
Cantidad no válida. Intente de nuevo.
1
Ingrese la fila (A-J): s
Fila no válida. Intente de nuevo.
Ingrese la fila (A-J): a
Ingrese la columna (0-9): e
Columna no válida. Intente de nuevo.
Ingrese la columna (0-9): 1
¡Reserva realizada con exito!

Presiona Enter para continuar...
```

9.5 Liberar Asientos

```
  LIBERAR ASIENTO
 Ingrese motivo de liberación del asiento
 1. Motivo de Salud
 2. Motivo de Viaje
 3. Motivo de Cambio de Planes
 4. Otro Motivo
 5
 Opción no válida. Se considerará como 'Otro Motivo'.
 Ingrese la cantidad de asientos a liberar
 2
 Ingrese la fila (A-J): a
 Ingrese la columna (0-9): 1
 ¡Asiento liberado con exito!
 Ingrese la fila (A-J): a
 Ingrese la columna (0-9): 0
 El asiento ya estaba libre

 Presiona Enter para continuar...
```

9.6 Estadísticas

```
ESTADiSTICAS DE LA SALA
Asientos Ocupados: 0
Asientos Libres:   100
Total de Asientos: 100
Porcentaje de Ocupación: 0%

Presiona Enter para continuar...
```

10 Explicación por fragmentos del código

10.1 Validación y lectura de coordenadas (LeerCoordenadas)

- Este método evita errores de ejecución si el usuario presiona Enter sin escribir datos o ingresa valores fuera de rango. Utiliza un arreglo de letras para convertir la fila ingresada (A-J) en su índice numérico correspondiente (0-9) y un ciclo while con int.TryParse para asegurar una columna válida:

```
    static (int fila, int columna) LeerCoordenadas()
{
    char[] letras = { 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J' };
    int fila = -1;
    int columna = -1;

    // Validación de la fila
    while (fila == -1)
    {
        Console.Write("Ingrese la fila (A-J): ");
        string entradaFila = Console.ReadLine().ToUpper();

        for (int j = 0; j < letras.Length; j++)
        {
            if (entradaFila.Length == 1 && letras[j] == entradaFila[0])
            {
                fila = j;
                break; // Si ya la encontró, no tiene caso seguir buscando
            }
        }

        if (fila == -1)
        {
            Console.WriteLine("Fila no válida. Intente de nuevo.");
        }
    }

    // Validación de la columna
    while (columna < 0 || columna > 9)
    {
        Console.Write("Ingrese la columna (0-9): ");

        if (!int.TryParse(Console.ReadLine(), out columna) || columna < 0 || columna > 9)
        {
            columna = -1;
            Console.WriteLine("Columna no válida. Intente de nuevo.");
        }
    }

    return (fila, columna);

    }
```

10.2 Reserva de asientos (ReservarAsiento)

Este método obtiene las coordenadas validadas mediante LeerCoordenadas() y evalúa el estado actual de esa posición en la matriz sala. Si el asiento está disponible ('O'), cambia su valor a ocupado ('X') y confirma la operación; de lo contrario, notifica que el lugar ya estaba ocupado:

```
static void ReservarAsiento()
{
    Console.WriteLine(" RESERVAR ASIENTO ");
    int cantidadAsientos = 0;
Console.WriteLine("Ingrese la cantidad de asientos a reservar");
// Validación de la cantidad de asientos a reservar (debe ser un número positivo y no mayor a 100)
while (!int.TryParse(Console.ReadLine(), out cantidadAsientos) || cantidadAsientos <= 0|| cantidadAsientos > 100)
{
    Console.WriteLine("Cantidad no válida. Intente de nuevo.");
}
// Se repite la lectura de coordenadas para la cantidad de asientos que el usuario desea reservar.
for (int o = 0; o < cantidadAsientos; o++)
{
    // Llamamos a LeerCoordenadas() para obtener la fila y columna del asiento a reservar.
var (fila, columna) = LeerCoordenadas();

// Si el usuario ingresó coordenadas inválidas, se sale del método.
if (fila == -1 || columna == -1)
{
    return;
}

// Se verifica si el asiento ya está ocupado. Si lo está, se informa al usuario; si no, se marca como ocupado.
if (sala[fila, columna] == 'X')
{
    Console.WriteLine("El asiento ya se encuentra ocupado");
}
else
{
    // Se marca el asiento como ocupado.
    sala[fila, columna] = 'X';
    Console.WriteLine("¡Reserva realizada con exito!");
}
}
}
```

10.3 Liberación de asientos (LiberarAsiento)

Al igual que en la reserva, reutiliza LeerCoordenadas() para obtener la ubicación exacta. Verifica si el asiento ya se encuentra disponible ('O'); si está ocupado ('X'), lo restablece a 'O' para dejarlo libre nuevamente:

```
    static void LiberarAsiento()
{
    string MotivoLiberacion = "";
    Console.WriteLine(" LIBERAR ASIENTO ");
    // Solicita al usuario que ingrese el motivo de liberación del asiento.
    Console.WriteLine("Ingrese motivo de liberación del asiento");
    Console.WriteLine("1. Motivo de Salud");
    Console.WriteLine("2. Motivo de Viaje");
    Console.WriteLine("3. Motivo de Cambio de Planes");
    Console.WriteLine("4. Otro Motivo");
    switch (Console.ReadLine())
        {
            case "1":
                MotivoLiberacion = "Motivo de Salud seleccionado.";
                break;
            case "2":
                MotivoLiberacion = "Motivo de Viaje seleccionado.";
                break;
            case "3":
                MotivoLiberacion = "Motivo de Cambio de Planes seleccionado.";
                break;
            case "4":
                Console.WriteLine("Ingrese el motivo de liberación del asiento:");
                MotivoLiberacion = "" + Console.ReadLine();
                break;
            default:
                Console.WriteLine("Opción no válida. Se considerará como 'Otro Motivo'.");
                break;
        }

            int cantidadAsientos = 0;
    // Solicita al usuario que ingrese la cantidad de asientos a liberar y valida la entrada.
    Console.WriteLine("Ingrese la cantidad de asientos a liberar");
    while (!int.TryParse(Console.ReadLine(), out cantidadAsientos) || cantidadAsientos <= 0|| cantidadAsientos > 100)
    {
        Console.WriteLine("Cantidad no válida. Intente de nuevo.");
    }
    // Se repite la lectura de coordenadas para la cantidad de asientos que el usuario desea liberar.
    for (int o = 0; o < cantidadAsientos; o++)
    {
        // Llamamos a LeerCoordenadas() para obtener la fila y columna del asiento a liberar.
    var (fila, columna) = LeerCoordenadas();
    // valida si el asiento esta libre
    if (sala[fila, columna] == 'O')
    {
        Console.WriteLine("El asiento ya estaba libre");
    }
    else
    {
        // Se marca el asiento como libre.
        sala[fila, columna] = 'O';
        Console.WriteLine("¡Asiento liberado con exito!");
    }
    }
}
// recorre la matriz e imprime la disponibilidad de los asientos seleccionados por el usuario.
```

10.4 Cálculo de estadísticas (MostrarEstadisticas)

Utiliza dos ciclos for anidados para recorrer las 100 posiciones de la matriz sala. Incrementa los contadores de asientos ocupados y libres, y realiza una división entre 100.0 (punto flotante) para conservar los decimales al calcular el porcentaje de ocupación:

```
static void MostrarEstadisticas()
{
    Console.WriteLine("ESTADiSTICAS DE LA SALA");

    int ocupados = 0;
    int libres = 0;
    // Recorre la matriz y cuenta el número de asientos ocupados y libres.
    for (int f = 0; f < 10; f++)
    {
        for (int c = 0; c < 10; c++)
        {
            if (sala[f, c] == 'X')
            {
                ocupados++;
            }
            else
            {
                libres++;
            }
        }
    }
    // Calcula el porcentaje de ocupación.
    double porcentaje = (ocupados / 100.0) * 100;
    // Muestra las estadísticas al usuario.
    Console.WriteLine($"Asientos Ocupados: {ocupados}");
    Console.WriteLine($"Asientos Libres:   {libres}");
    Console.WriteLine($"Total de Asientos: 100");
    Console.WriteLine($"Porcentaje de Ocupación: {porcentaje}%");
}
}
```

11. Conclusiones individuales

- **Roberto Adan Patiño Rayo:** El desarrollo de este proyecto me permitió comprender cómo aplicar arreglos bidimensionales (matrices) en C# para modelar problemas reales como la distribución de una sala de cine. Asimismo, aprendí la importancia de validar las entradas del usuario con int.TryParse y ciclos while para evitar cierres inesperados del programa, además de gestionar el trabajo colaborativo y el control de versiones mediante Git y GitHub en Visual Studio Code.
- **Gadi Jeani Del Carmen Memije:** A través de la implementación modular del sistema, pude reforzar el uso de ciclos anidados, estructuras condicionales y funciones con retorno de valores. Dividir el problema en diagramas de flujo antes de programar facilitó mucho la escritura de un código limpio, ordenado y fácil de depurar en equipo.

12. Referencias

- Microsoft Learn. (2026). *Matrices multidimensionales (Guía de programación de C#)*. Microsoft Documentation. https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/builtin-types/arrays
- Microsoft Learn. (2026). *Int32.TryParse Método (System)*. Microsoft Documentation. https://learn.microsoft.com/es/dotnet/api/system.int32.tryparse
- GitHub Docs. (2026). *Documentación sobre control de versiones con Git y GitHub*. https://docs.github.com/es